Packaging for Unified Origin

Table of Contents

- Package MP4 to fragmented-MP4 and back
- How to package CMAF
- Options for fragmented-MP4 packaging
- Overriding and adding track properties
- Packaging content for delivery by Unified Origin

Package MP4 to fragmented-MP4 and back

Fragmented-MP4 files such as PIFF, ISMV or CMAF can be packaged using Unified Packager for use with Unified Origin.

There is a specific section on How to package CMAF.

Converting from an MP4 file to a fragmented mp4 file (.ismv) can be done as follows:

```
#!/bin/bash

mp4split -o tears-of-steel-avc1-750k.ismv \
  tears-of-steel-avc1-750k.mp4
```

The other way around, from fragmented MP4 / PIFF / ISMV / CMAF to MP4:

```
#!/bin/bash

mp4split -o tears-of-steel-avc1-750k.mp4 \
  tears-of-steel-avc1-750k.ismv
```

Or from Adobe's F4F to MP4:

```
#!/bin/bash

mp4split -o tears-of-steel-avc1-750k.mp4 \
  tears-of-steel-avc1-750k.f4f
```

You can also use the server manifest file as input. All the audio and video streams referenced in the manifest file are combined into one MP4 file.

```
#!/bin/bash

mp4split -o tears-of-steel-avc1-750k.mp4 \
  tears-of-steel-avc1.ism
```

It is also possible to package multiple tracks into a single MP4. When doing this, the order in which the tracks are specified on the command-line will define the order of the tracks within the MP4 (which can be useful for Define default track when preparing content (track order)):

```
#!/bin/bash

mp4split -o tos_avc1-sorted.ismv \
  tears-of-steel-avc1-1000k.mp4 \
  tears-of-steel-avc1-1500k.mp4 \
  tears-of-steel-avc1-750k.mp4 \
  tears-of-steel-avc1-400k.mp4
```

How to package CMAF

New in version 1.8.3.

The Common Media Application Format is defined in ISO/IEC 23000-19, and is a preferred interopable profile for using fragmented MP4.

The process of packaging CMAF is very similar to using `mp4split` to package ISMV. Depending on the extension of the output, `mp4split` will decide how the content is packaged. In the case of CMAF, you can choose between `.cmfv` for video, `.cmfa` for audio and `.cmft` for text streams.

The command-line that you use is very straightforward, as shown in the examples for packaging CMAF audio, video an text streams video below:

```bash
#!/bin/bash

mp4split -o tears-of-steel-aac-128k.cmfa tears-of-steel-aac-128k.mp4
mp4split -o tears-of-steel-avc1-2200k.cmfv tears-of-steel-avc1-2200k.mp4
mp4split -o tears-of-steel-en.cmft tears-of-steel-en.srt
```

Use tos-cmaf-packaging.sh to package the Tears of Steel demo contents to CMAF.

If you are packaging CMAF to support fMP4 HLS and want to follow Apple's recommendation of delivering 6 second segments, you will have to make sure that the CMAF files that contain your media are fragmented with this length in mind as each fragment in a MP4 equals a 'play-out' segment when content is statically packaged (when using Origin, the segment length can be changed on-the-fly for each play-out format).

You can make sure your content is fragmented to your liking by using the `--fragment_duration` option while packaging CMAF (per default, *Unified Packager* fragments the MP4 according to the GOP structure of the media). See below for an example.

However, do note that the specified length should be a multiple of the media's GOP length, as segments can only contain full GOPs. So, in case you are ingesting media with 1.92 seconds GOPs and want to aim for a segment duration of 6 seconds, you should specify a fragment duration of 576/100.

In below example GOP length is 48 frames at 24Hz, or 2 seconds. We use 4 GOPs per fragment to align segment boundaries with audio (because 375 blocks of 1024 samples at 48kHz is precisely 8 seconds):

```bash
#!/bin/bash

mp4split -o tos-8s-aac-128k.cmfa \
  --fragment_duration=384000/48000 \
  tears-of-steel-aac-128k.mp4
mp4split -o tos-8s-avc1-2200k.cmfv \
  --fragment_duration=192/24 \
  tears-of-steel-avc1-2200k.mp4
mp4split -o tos-8s-en.cmft \
  --fragment_duration=8/1 \
  tears-of-steel-en.srt
```

Note

If you want to override or add any properties to a track when packaging CMAF, you can do so using the familiar options: Overriding and adding track properties.

> **Note**
>
> Fragmented-MP4 based on CMAF, according to the specification only a single track per file is suppored. Thus in case you input contains multiple tracks of the same content type it is necesary to select the track using --track_id option.

Options for fragmented-MP4 packaging

The packager supports the following options.

--use_dref

Create a (progressive) mp4 that references a fragmented mp4 file, for "progressive download" to older players. *Unified Origin* will resolve media data references on playout.

The path(s) to the input you provide on the command-line can be relative or absolute. If a relative path is used, it must be a downward path and the input must be local.

To create a dref that references remote content using relative paths, use the --dref_base_url option, or mount the remote content so that you process it as if it's local. You can do this using a tool like `s3fs` or similar.

--use_dref_no_subs

New in version 1.7.27.

Like "--use_dref" creates mp4 that references an mp4 file, but without explicitly referencing sub-samples, resulting in a (considerably) smaller video mp4.

The path(s) to the input you provide on the command-line can be relative or absolute. If a relative path is used, it must be a downward path and the input must be local. To create a dref that references remote content using relative paths, mount the remote content so that you process it as if it's local. You can do this using a tool like `s3fs` or similar.

> **Note**
>
> If you want offer a download to own option use --use_dref, because encryption requires sub-sample data. However, for progressive download without encryption --use_dref_no_subs suffices.

--dref_base_url

New in version 1.15.20.

To create a dref that references remote content using relative paths, you can use the `--dref_base_url` option, which specifies the base URL used to determine the relative location of the remote content.

For example, if remote content is located at http://remote.example.com/some/path/subdir1/content1.mp4 and http://remote.example.com/some/path/subdir2/content2.mp4, and you want to create a local dref that uses the relative paths `subdir1/content1.mp4` and `subdir2/content2.mp4` as references, you run:

```
#!/bin/bash

mp4split -o dref.mp4 --use_dref \
  --dref_base_url=http://remote.example.com/some/path/ \
  http://remote.example.com/some/path/subdir1/content1.mp4 \
  http://remote.example.com/some/path/subdir2/content2.mp4
```

The "common" part, http://remote.example.com/some/path/, a.k.a. the base URL, is removed from the references, and the resulting local file `dref.mp4` will have relative references to `subdir1/content1.mp4` and `subdir2/content2.mp4`.

Afterwards you upload the local file `dref.mp4` to the remote location via other means, so that it is accessible at the URL http://remote.example.com/some/path/dref.mp4.

--dry_run

Do not write the output.

--timescale

The output timescale used for media. Defaults to the original media or 10MHz when the "piff" brand is used.

--fragment_duration

The target duration of each fragment expressed as a fraction "X/Y" of seconds (or in milliseconds "X"), default 2s. Behaviour is similar to `#EXT-X-TARGETDURATION` in HLS or `maxSegmentDuration` in MPEG-DASH. When sync-samples are present, each fragment starts with a sync-sample and has 0 or more additional sync samples - as many as will fit into *fragment duration*.

This parameter can be useful to align fragment boundaries across different codecs or tuning the fragment duration for a specific playout format. For example, Apple recommends 6 second segments in HLS, while 2 seconds is common in Smooth Streaming.

--brand

Sets the 'brand'. Common options: "piff", "iso6", "ccff", "dash" and "cmfc". Default is "iso6", but with timescale=10000000 (10Mhz) the default is "piff". When packaging CMAF (i.e., extension of output is .cmfv, .cmfa or .cmft) the default is "cmfc".

When creating (progressive) mp4 files with negative-composition-times "iso4" is used as brand. When using "iso2" negative composition time offsets are disabled and an edit list is used to compensate for the ct_offset.

By using the `--brand` option, you overrule the default major brand for the given output. This can be helpful if you want to make sure that your output uses negative composition time offsets instead of an edit list ("iso4") or, the other way around, uses an edit list instead of negative composition time offsets ("iso2"). When using the option more than once on the same command-line, any brands specified after the first will be added as compatibility brands.

For instance, the first example below will result in a CMAF-file with "cmfc" as its major brand and "iso9" as a compatibility brand, whereas the compatibility brand will be "dash" in the second example:

```
mp4split -o example.cmfv --brand=cmfc --brand=iso9 example.mp4
mp4split -o example.cmfv --brand=cmfc --brand=dash example.mp4
```

--timestamp_offset

If your workflow for fMP4 HLS involves one or more WebVTT files that are or were part of a HLS Transport Stream playout scenario, chances are that a 10 seconds time offset is signaled in the WebVTT (using the `EXT-X-TIMESTAMP-MAP` tag).

If this is the case, it will result in the subtitles being out of sync as the default when packaging CMAF is to not use a time offset. To synchronize the WebVTT timeline with the other media you can simply remove the `EXT-X-TIMESTAMP-MAP` tag from the WebVTT file.

Synchronizing the other media to the WebVTT timeline is also possible, but not recommended. To do so, offset all other media when packaging by adding the option `--timestamp_offset=10`. The following can also be used when re-packaging HLS-TS, but this is not recommended.

--positive_composition_offsets

Whenever video media samples are reordered a composition delay is introduced. To compensate for this delay we use negative composition offsets (version 1 "ctts" and "trun" boxes) where necessary.

You can also use positive composition offsets (version 0 "ctts" and "trun" boxes). An edit list is then added to remove the composition delay. Note that the use of this option is not recommended.

--source_description

Allows you to provide a MPD (DASH manifest) to define the exact fragment boundaries when packaging specific input. This can be used if a fixed fragment duration across the whole presentation is not flexible enough.

See Packaging using an MPD Source Description for more information.

Overriding and adding track properties

When generating a fragmented or progressive MP4 file (.mp4, .isma, .ismv or .ismt) from an input track, its track properties are based on the properties of the input track. It is possible to add or override some of these properties, but in most cases, this is not necessary.

> **Note**
>
> You can also set a name and description for each track, but that is only possible when generating a server manifest. See --track_name and --track_description.

When generating a fragmented or progressive MP4, the track properties that can be added or overridden are the following:

--track_language

By default *track_language* is taken from the input track's media info. In case you do need or want to set the language for a track, make sure to use the correct RFC 5646 language tags. These language tags consist of two-letter, three-letter, extended languages, and scripts.

> **Note**
>
> Only DASH and HLS offer support for RFC 5646. For output formats that do not support it, a tag's first two or three characters are parsed according to ISO 639-1 or ISO 639-2/T. If this does not result in a valid language tag, `und` is used. To make sure that a valid fallback option is available, it is good practice to specify a macro language when possible. For example: signal Cantonese Chinese using `zh-yue` rather than `yue` so that 'Chinese' is used as the fallback option.

For example, specifying languages with two letter ISO 639-1 language codes:

- `en` for English
- `nl` for Dutch
- `es` for Spanish

For languages that do not have two letter language codes but do have ISO 639-2/T or ISO 639-3 codes:

- `haw` for Hawaiian
- `yue` for Cantonese

For languages as used in different regions:

- `en-UK` for English as used in the UK
- `nl-BE` for Dutch; Flemish as used in Belgium
- `pt-BR` for Portuguese as used in Brazil

For additional scripting tags for languages:

- `sr-Cyrl` for Serbian using the Cyrillic script
- `zh-Hans` for Chinese using the simplified script

> **Note**

In addition to the language tag, HLS requires the presence of a language name. For tags that are part of ISO 639-1 or ISO 639-2/T mapping of the tag to a name is automatic. For all other language tags, --track_description should be used to signal the name.

```
#!/bin/bash

mp4split -o audio-en.mp4 --track_language=en
mp4split -o audio-nl-be.mp4 --track_language=nl-be \
            --track_description="Vlaams Nederlands"
mp4split -o audio-zh-yue-hant.mp4 --track_language=zh-yue-hant \
            --track_description="Cantonese Chinese using Traditional script"
```

Language tag formatting

New in version 1.10.2.

Note

As defined in the RFC 5646 Formatting, the capitalization of language tags is now enforced. Thus nl-be will be formatted to nl-BE, zh-hans will be formatted to zh-Hans, and EN-US will be formatted to en-US.

--track_bitrate

Overrides the average bitrate of a track.

By default *track_bitrate* is the average bitrate (either from the metadata info of the input track, or calculated from the source samples). But it can be overridden explicitly like the following:

```
#!/bin/bash

mp4split -o tos-override-bitrates.ismv \
 tears-of-steel-aac-64k.mp4 --track_bitrate=64000 \
 tears-of-steel-aac-128k.mp4 --track_bitrate=128000 \
 tears-of-steel-avc1-400k.mp4 --track_bitrate=400000 \
 tears-of-steel-avc1-750k.mp4 --track_bitrate=750000 \
 tears-of-steel-avc1-1000k.mp4 --track_bitrate=1000000
```

You can also set this to `max`, so that the maximum/peak bitrate is used. In case the source video contains audio as well then audio and video should be processed in separate steps to get the wanted values:

```bash
#!/bin/bash

mp4split -o video.ism Origin.mp4 --track_type=video --track_bitrate=max
mp4split -o audio.ism Origin.mp4 --track_type=audio
mp4split -o main.ism audio.ism video.ism
```

Resulting manifest will have the following, with the max video bitrate taken from the source content:

```
<audio src="Origin.mp4" systemBitrate="96000" systemLanguage="por">
...
<video src="Origin.mp4" systemBitrate="3155800">
```

Note

To determine the max bitrate the complete track is parsed and bitrate is calculated for each second of media data in the track. The highest value of all of the calculated values is considered the max bitrate of the track.

--track_role

Sets the role of a track and can be used to further distinguish it, next to bitrate and language. The exact meaning of a role can be dependent on the kind of track it is added to (video, audio or text). All of the roles specified in urn:mpeg:dash:role:2011 can be used. Most of them are listed in the table below:

--track_role=	Description
main	main media intended for presentation if no other information is provided.
alternate	media that is an alternative to the main media of the same type.
supplementary	media that is supplementary to media content of a different media component type.
commentary	media content component with commentary.
caption	media content component with captions (typically containing description of music an
subtitle	media content component with subtitles.
description	track containing textual description (intended for audio synthesis) or audio descriptic
metadata	media component containing information intended to be processed by application sp
forced-subtitle	textual information meant for display when no other text representation is selected.

```bash
#!/bin/bash

mp4split -o example-audio.isma \
  example-audio.mp4 --track_role=main --track_language=eng
mp4split -o example-commentary.isma \
  example-commentary.mp4 --track_role=alternate --track_language=eng
```

Attention

Never mix the use of --track_role and --track_kind when you want to enable the signaling of accessibility features for a track.

If you want to signal the accessibility features for an audio description track in DASH and HLS, using --track_role to define the track's role as 'description' will not get you the results that you expect (it will only affect the DASH output, and only change the track's role, not add an `<Accessibility>` element). Please read the section on the --track_kind option below, including the 'Use case walkthrough' that explains how to add this signaling step-by-step.

Adding accessibility signaling for a captions track is more straightforward, as it only requires you to specify the track's role as `caption` when packaging it, or when creating the server manifest (using `--track_role=caption`).

The signaling for both captions and audio description tracks is based on the DVB-DASH specification for DASH and the HLS Authoring Specification for HLS.

In addition to the signaling described above, it's also possible to trigger logic that will add the 'FORCED=YES' attribute and value to a subtitles track in HLS by specifying the track's role as `forced-subtitle` (this will also add the 'forced-subtitle' role for the track in DASH, which is the default behavior for DASH when a role is specified for a subtitles track).

--track_kind

New in version 1.7.31.

Adds a SchemeIdUri/Value pair to the 'kind' box when packaging a (f)MP4. This box should describe the intended purpose of the track. Similar to the --track_role option described above, the --track_kind option can be used to further distinguish a track, besides its bitrate and language.

Specifying the parameters of this option is done like so:

```
--track_kind="<SchemeIdUri>@<Value>"
```

Where the <SchemeIdUri> and <Value> should be replaced with parameters of choice, preferably from the `about:html-kind` defined by W3C HTML5 or the `urn:mpeg:dash:role:2011` scheme defined by MPEG-DASH (although the latter can be signaled more easily using the --track_role option).

When packaging for *Unified Origin*, the main use case of the `--track_kind` option is adding and properly signaling tracks that provide accessibility features, such as captions for the hard of hearing or an audio description of the video track for the visually impaired. The first can be signaled using `urn:mpeg:dash:role:2011@caption` (or `--track_role=caption`), whereas the `about:html-kind` can be used to signal the latter with `about:html-kind@main-desc`.

Take for example the situation in which the `about:html-kind@main-desc` 'kind' is present in a track that has been added to a server manifest. *Unified Origin* will then add the following parameters for this track when generating a DASH client manifest (.mpd) and HLS main playlist (.m3u8) for playout, based on the DVB-DASH specification and the HLS Authoring Specification respectively:

MPEG-DASH (.mpd)

```
<Accessibility
  schemeIdUri="urn:tva:metadata:cs:AudioPurposeCS:2007"
  value="1">
</Accessibility>
<Role
    schemeIdUri="urn:mpeg:dash:role:2011"
    value="alternate">
</Role>
```

HLS (.m3u8)

```
CHARACTERISTICS="public.accessibility.describes-video",AUTOSELECT=YES
```

Configuring Audio description track how to

As an example, consider as a starting point:

- ABR video in `english-video.ismv`

- Main audio in `english-audio.isma`
- Alternate audio in `welsh-audio.isma`
- Audio description in `english-ad-without-kind-box.isma`

Assuming the other tracks are packaged correctly, only the audio description track needs to be repackaged (to include the 'kind' box with the accessibility info). Because its language, bitrate and codec is identical to the main audio track, the 'kind' box is what distinguishes it (apart from its actual content of course).

As indicated above, `about:html-kind@main-desc` should be used as the value for the kind box for audio description tracks, so:

```
#!/bin/bash

mp4split -o english-ad-with-kind-box.isma \
  english-ad-without-kind-box.isma \
  --track_kind="about:html-kind@main-desc"
```

After repackaging the audio description track, generate a server manifest to stream with *Unified Origin*:

```
#!/bin/bash

mp4split -o presentation.ism \
  --hls.client_manifest_version=4 \
  english-video.ismv \
  english-audio.isma \
  welsh-audio.isma \
  english-ad-with-kind-box.isma
```

Or use the tracks for a static packaging workflow using *Unified Packager* to create DASH or HLS streams (accessibility signaling is not supported for Smooth).

Packaging content for delivery by Unified Origin

The first step is to package all the source content into the format that is used by *Unified Origin*. This is the fragmented-MP4 format.

The example uses this Source Content.

```bash
#!/bin/bash

mp4split -o video_400k.ismv \
  tears-of-steel-avc1-400k.mp4 \
  tears-of-steel-aac-64k.mp4

mp4split -o video_800k.ismv \
  tears-of-steel-avc1-750k.mp4 \
  tears-of-steel-he-aac.mp4

mp4split -o video.ismv \
  tears-of-steel-avc1-400k.mp4 \
  tears-of-steel-avc1-750k.mp4 \
  tears-of-steel-dts-384k.mp4 \
  tears-of-steel-ac3-448k.mp4 \
```

Now that we have packaged all the audio and video, the following step is to create the two progressive download files. Instead of creating a completely new MP4 video file we will create an MP4 video that only contains the necessary index and references the actual movie data that is stored in the fragmented-MP4 format.

```bash
#!/bin/bash

mp4split -o video_400k.mp4 --use_dref \
  video_400k.ismv
```

```bash
#!/bin/bash

mp4split -o video_800k.mp4 --use_dref \
  video_800k.ismv
```

As a last step we create the server manifest file. This is an XML file that contains the media information about all the tracks and is used by the USP webserver module.

```bash
#!/bin/bash

mp4split -o video.ism \
  video.ismv \
  video_400k.ismv \
  video_800k.ismv
```

At this point we have six files stored for our presentation:

File	Description
video_400k.ismv	AAC-LC, 400 kbps video
video_800k.ismv	HE-AAC, 800 kbps video
video.ismv	200/600 kbps video, DTS, AC3
video_400k.mp4	AAC-LC, 400 kbps video
video_800k.mp4	HE-AAC, 800 kbps video
video.ism	USP server manifest file

The USP webserver module makes the following URLs available. Note that except for the progressive download URLs, they are all virtual and do not exist on disk:

Playout format	URL
Smooth Streaming	http://www.example.com/usp/video.ism/Manifest
HTTP Live Streaming	http://www.example.com/usp/video.ism/video.m3u8
HTTP Dynamic Streaming	http://www.example.com/usp/video.ism/video.f4m
MPEG-DASH	http://www.example.com/usp/video.ism/video.mpd
Progressive download	http://www.example.com/usp/video_400k.mp4
Progressive download	http://www.example.com/usp/video_800k.mp4

Please download the advanced-usp.sh sample script which creates the various server manifest as discussed above. The sample content is Tears of Steel.